Implementation Tasks: Integrate All Backend Features into Frontend

Feature: Integrate All Backend Features into Frontend
Branch: `006-backend-features-full-integration`
Created: 2026-02-05

Implementation Strategy

This implementation will deliver all backend features into the frontend UI in a phased approach, with each user story being independently testable. The approach prioritizes delivering value early by implementing the highest priority features first (advanced search and filtering, real-time notifications), followed by the AI chat interface, and finally the analytics dashboard.

Dependencies

User stories are designed to be largely independent but share foundational components:

- User Story 2 (Real-time Notifications) depends on foundational WebSocket setup from foundational tasks
- User Story 3 (AI Chat Interface) depends on foundational components
- User Story 4 (Analytics Dashboard) can be implemented independently

Parallel Execution Examples

Per User Story 1 (Advanced Search & Filtering):

- [P] Create searchService.ts with full-text search functionality
- [P] Create SearchFilterPanel.tsx component with advanced filters
- [P] Update TaskManager.tsx to integrate search functionality

Per User Story 2 (Real-time Notifications & Reminders):

- [P] Create notificationService.ts with WebSocket integration
- [P] Create NotificationPanel.tsx component
- [P] Create useNotifications.ts hook

Per User Story 3 (AI-Powered Chat Interface):

- [P] Create chatService.ts for chat functionality
- [P] Create ChatInterface.tsx component
- [P] Create useWebSocket.ts hook

Per User Story 4 (Advanced Task Analytics & Insights):

- [P] Create AnalyticsDashboard.tsx component
- [P] Update taskService.ts to include analytics endpoints

Phase 1: Setup

- [x] T001 Set up development environment and verify project structure matches plan requirements
- [x] T002 Install necessary dependencies for WebSocket and real-time features
- [x] T003 Verify backend API connectivity and test endpoints

Phase 2: Foundational Components

- [x] T004 [P] Update taskService.ts to support all new API parameters (search, filters, sorting)
- [x] T005 [P] Update TypeScript interfaces in taskService.ts to include new search result types
- [x] T006 [P] Create WebSocket service for real-time communication in frontend/services/webSocketService.ts
- [x] T007 [P] Update TaskManager.tsx to accept new search and filter parameters and state management
- [x] T008 [P] Update taskTypes.ts to include new entity types from data model

Phase 3: User Story 1 - Advanced Search & Filtering (Priority: P1)

Goal: As a user, I want to search and filter my tasks using natural language queries and advanced filters so that I can quickly find relevant tasks without manually scanning through all of them.

Independent Test: Can be fully tested by enabling users to enter natural language queries and apply various filters to see relevant tasks returned.

Implementation Tasks:

- [x] T009 [P] [US1] Create searchService.ts in frontend/services/ to handle full-text search and natural language parsing
- [x] T010 [P] [US1] Create SearchFilterPanel.tsx component in frontend/components/tasks/ with advanced filtering UI
- [x] T011 [US1] Update TaskManager.tsx to integrate search functionality and filter controls
- [x] T012 [US1] Implement search suggestions functionality with useSearchSuggestions.ts hook
- [x] T013 [US1] Test advanced search and filtering functionality end-to-end

Phase 4: User Story 2 - Real-time Notifications & Reminders (Priority: P1)

Goal: As a user, I want to receive real-time browser notifications for task reminders and important updates so that I stay informed about upcoming deadlines and task changes without constantly checking the app.

Independent Test: Can be tested by configuring reminder settings and verifying that browser notifications appear at the appropriate times.

Implementation Tasks:

- [x] T014 [P] [US2] Create notificationService.ts in frontend/services/ with WebSocket integration
- [x] T015 [P] [US2] Create NotificationPanel.tsx component in frontend/components/tasks/ for displaying notifications

- [x] T016 [P] [US2] Create useNotifications.ts hook in frontend/hooks/ for managing notification state
- [ ] T017 [US2] Integrate real-time notifications into TaskManager.tsx
- [ ] T018 [US2] Test real-time notification functionality with WebSocket connections

Phase 5: User Story 3 - AI-Powered Chat Interface (Priority: P2)

Goal: As a user, I want to interact with an AI assistant through a chat interface to manage tasks using natural language so that I can quickly create, update, or find tasks without navigating through multiple screens.

Independent Test: Can be tested by engaging with the chat interface and verifying that AI understands natural language commands and performs appropriate task operations.

Implementation Tasks:

- [x] T019 [P] [US3] Create chatService.ts in frontend/services/ for handling chat messages and AI processing
- [x] T020 [P] [US3] Create ChatInterface.tsx component in frontend/components/tasks/ with conversational UI
- [x] T021 [P] [US3] Create useWebSocket.ts hook in frontend/hooks/ for managing WebSocket connections
- [x] T022 [US3] Integrate chat interface into TaskManager.tsx
- [ ] T023 [US3] Test AI chat interface functionality with natural language commands

Phase 6: User Story 4 - Advanced Task Analytics & Insights (Priority: P3)

Goal: As a user, I want to view analytics and insights about my task completion patterns and productivity so that I can understand my habits and improve my task management.

Independent Test: Can be tested by viewing the analytics dashboard and verifying that it displays meaningful data about task completion patterns.

Implementation Tasks:

- [x] T024 [P] [US4] Create AnalyticsDashboard.tsx component in frontend/components/tasks/ with charts and metrics
- [x] T025 [P] [US4] Update taskService.ts to include analytics endpoints and data retrieval
- [x] T026 [US4] Integrate analytics dashboard into TaskManager.tsx
- [x] T027 [US4] Test analytics dashboard functionality with real data

Phase 7: Polish & Cross-Cutting Concerns

- [x] T028 [P] Add proper error handling for all new API calls and WebSocket connections
- [x] T029 [P] Add loading states for all new asynchronous operations (search, notifications, chat)
- [x] T030 Update documentation and comments for new functionality
- [x] T031 Perform end-to-end testing of all features together
- [x] T032 Optimize performance for search operations and real-time updates

- [x] T033 Clean up any temporary code or debugging elements
- [ ] T034 Verify all functionality works offline as per requirements